SWE 4538: SERVER PROGRAMMING

Lab 01: Introduction to Server Programming

Building Your First REST API with FastAPI

Language: Python | **Framework:** FastAPI

---

1 Overview

In this lab, you will build a small web server and explore how clients communicate with servers using HTTP. You will then create a simple API for **E-Shop**, an e-commerce application that will be expanded in future labs.

2 Learning Outcomes

By the end of this lab, you should be able to:

1. Explain the roles of a client, server, HTTP request, and HTTP response.
2. Set up and run a FastAPI development server.
3. Create routes that handle GET and POST requests.
4. Work with path parameters, query parameters, and JSON data.
5. Test API endpoints using FastAPI's interactive documentation.
6. Explain why data stored in a Python list disappears when a server restarts.

3 Before You Start

- **Prerequisites:** Basic Python knowledge (functions, lists, dictionaries, loops, and conditionals).
- **Required tools:** Python 3.10+, a code editor (e.g., VS Code), a terminal, and a web browser.

3.1 Quick Concepts

- **Client:** A program (such as your browser) that sends a request.
- **Server:** A program that receives requests and sends responses.
- **Endpoint/route:** A URL path handled by the server (for example, `/products`).
- **GET:** Requests information from a server.
- **POST:** Sends data to a server, often to create something.
- **JSON:** A common text format for exchanging structured data.

3.2 Part A: Guided Setup and First API (We Do)

Task A0 – Check your Python Installation

1. Open Command Prompt or Bash and run:

```
python --version
```

2. Install python if needed from the official distribution. Run the installer, ensuring Python is available from the terminal (on installers with the option, select **Add Python to PATH**).

Task A1 – Set up your workspace.

1. Create a folder named `eshop-lab1` and open it in your editor.
2. Open a terminal in this folder and create a virtual environment:

```
python -m venv .venv
```

Activate the environment using **one** of the commands below:

```
# Windows PowerShell
.venv\Scripts\Activate.ps1

# Windows Command Prompt
.venv\Scripts\activate.bat

# macOS / Linux
source .venv/bin/activate
```

3. Install FastAPI:

```
python -m pip install "fastapi[standard]"
```

Task A2 – Create your first server. Create `main.py` with the following code:

```python
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
def home():
    return {"message": "Welcome to Server Programming!"}
```

Start the development server:

```
fastapi dev main.py
```

Visit `http://127.0.0.1:8000/` in your browser. Then visit `http://127.0.0.1:8000/docs` to see the interactive API documentation. Keep the server running while completing the tasks; use `Ctrl+C` to stop it when necessary.

4 Part B: Routes and Parameters (We Do)

Task B1 – Create a personalized greeting. Add the following route to `main.py`:

```python
@app.get("/hello/{name}")
def get_hello(name: str):
    return {"message": f"Hello, {name}!"}
```

Test different names in the browser. Observe how the value of `name` changes.

Task B2 – Practice independently. Create a GET endpoint `/square/{number}` that accepts an integer and returns its square. For example:

```
GET /square/5

# Expected JSON:
{"number": 5, "square": 25}
```

5 Part C: Your First E-Shop API (You Do)

The E-Shop application needs a product catalog. For now, store the products directly in memory by placing this list in `main.py`, below the imports and `app` creation:

```
products = [
    {"id": 1, "name": "Laptop", "price": 75000},
    {"id": 2, "name": "Mouse", "price": 800},
    {"id": 3, "name": "Keyboard", "price": 1500},
]
```

Populate the list with at least 10 entries.

5.1 Task C1 – List all products

Implement a `GET /products` endpoint that returns the entire `products` list. The response should be a JSON array containing the three starting products.

5.2 Task C2 – Get one product by ID

Implement a `GET /products/{product_id}` endpoint. It should:

1. Accept `product_id` as an integer path parameter.
2. Search the `products` list for the matching ID.
3. Return the matching product as JSON.
4. Return HTTP status **404 Not Found** when the ID does not exist.

Example:

```
GET /products/2
Expected JSON: {"id": 2, "name": "Mouse", "price": 800}
```

For a missing product, FastAPI provides an exception class:

```
from fastapi import HTTPException

# Use this inside your endpoint if no matching product is found:
raise HTTPException(status_code=404, detail="Product not found")
```

Important: The full route `/products` and the parameterized route `/products/{product_id}` are different endpoints.

6 Part D: Accepting Data with POST (You Do)

A GET request retrieves information. To let a client *create* a product, we need a POST request with a JSON request body.

6.1 Task D1 – Define the request structure

Add the following import and class to `main.py`:

```
from pydantic import BaseModel

class ProductCreate(BaseModel):
    name: str
    price: float
```

This model describes the fields that the client must send. The server will generate the new product ID, so the client does not need to provide it.

6.2 Task D2 – Implement POST /products

Implement `POST /products` so that it:

1. Receives a request body matching `ProductCreate`.
2. Assigns a unique integer ID (for this lab, use one greater than the current maximum ID; start at 1 if the list is empty).
3. Adds the new product to the `products` list.
4. Returns the newly created product, including its assigned ID.

Test in Swagger UI: Open `http://127.0.0.1:8000/docs`, expand `POST /products`, click **Try it out**, and submit:

```
{
  "name": "Monitor",
  "price": 12000
}
```

Your response should contain the new product with its generated ID. Then call `GET /products` again and verify that the new product appears.

7 Part E: Filtering and Search

Complete these only after Parts A–D work correctly.

1. **Search by name:** Implement `GET /products/search?name=mouse` to return products whose names contain the supplied text, ignoring capitalization.
2. **Filter by price:** Implement `GET /products/affordable?max_price=2000` to return products with prices less than or equal to the given value.

Routing note: Define fixed paths such as `/products/search` and `/products/affordable` *before* `/products/{product_id}`, so the parameterized route does not capture the words `search` or `affordable`.